

Task Tracker Patch Exercise: Working Notes

Repo: github.com/thelakshrao/superrdev-patch-Assignment | Bugs covered so far: Bug 1 to Bug 7

What the app does

The Task Tracker shows a list of 49 tasks with a search box and a status dropdown. Whenever I type or pick a status, React sends a request to the backend (GET `/api/tasks?q=...&status=...`). TaskController receives it and asks TaskRepository, which runs a SQL query on the H2 database. The database sends back the matching rows and they appear in the table. So a mistake in the SQL shows wrong rows even if React and Spring work correctly, a delay in the controller slows every request, and a mistake in the React hook can show results in the wrong order.

Summary of fixes

#	Bug	Where	Result
1	SQL AND/OR precedence: status filter ignored, archived tasks leaked in	TaskRepository.java (also db/queries and Oracle package) - SQL layer	Status = Done: total 49 changed to 5
2	Artificial Thread.sleep delayed short and empty searches by up to 1 second	TaskController.java, lines 35-42 - Java layer	Empty search: 1.025 s changed to 0.008 s
3	Search fired a request on every keystroke and old responses could overwrite new ones	frontend/src: useTasks.js, api.js, App.jsx, new useDebounce.js - React layer	Typing "api": 3 requests changed to 1
4	"Loading tasks..." shown forever when the backend is down; old error never cleared	frontend/src/hooks/useTasks.js (.catch block) - React layer	Error message now shown; recovers when backend returns
5	Page number not reset when search or status changes: empty page, no way back	frontend/src/App.jsx - React layer	Done from page 3: page=3 (0 rows) changed to page=1 (5 rows)

How I worked

For each bug I first reproduced it (browser Network tab and curl), found the cause by reading the code, made the smallest fix, restarted and repeated the same test to compare before and after. I committed each fix separately. Tool used: Claude.ai, to help me understand the problems and concepts; I ran every test and applied every change myself.

Bug 1: SQL AND/OR Precedence

Commit: "Fix SQL AND/OR precedence in task search" (7f69cc0), 3 files changed

Where the bug is

backend/src/main/java/com/internal/tasktracker/TaskRepository.java, the @Query on lines 14-16 (SQL layer). The same mistake was copied into db/queries/search_tasks.sql and twice into db/oracle/task_search_package.sql (the COUNT query and the cursor query).

How I found it

I selected Done in the dropdown and opened Chrome DevTools, Network tab. The request was `tasks?status=DONE&page=1&pageSize=10` (React sent the right value), but the response said **total: 49** and the table showed OPEN tasks (ids 49, 48, 46). The correct answer is 5 Done tasks. Since the request was correct, I looked at the SQL.

Root cause

In SQL, AND is evaluated before OR (like multiplication before addition). Without brackets the query became two separate rules joined by OR:

```
WHERE archived = FALSE AND LOWER(title) LIKE :term
OR LOWER(description) LIKE :term AND (:status IS NULL OR status = :status)
```

Rule 1 = (not archived AND title matches), which has no status check. Rule 2 = (description matches AND status matches), which has no archived check. With an empty search every title matches, so Rule 1 lets in all 47 non-archived tasks whatever their status; Rule 2 adds the 2 archived Done tasks. $47 + 2 = 49$, so the status filter looked dead and archived tasks leaked in.

Fix and why

I put brackets around the OR so that "title or description matches" is one condition and all three filters must be true together. It is the smallest possible change and keeps the same logic.

```
WHERE archived = FALSE
AND (LOWER(title) LIKE :term OR LOWER(description) LIKE :term)
AND (:status IS NULL OR status = :status)
```

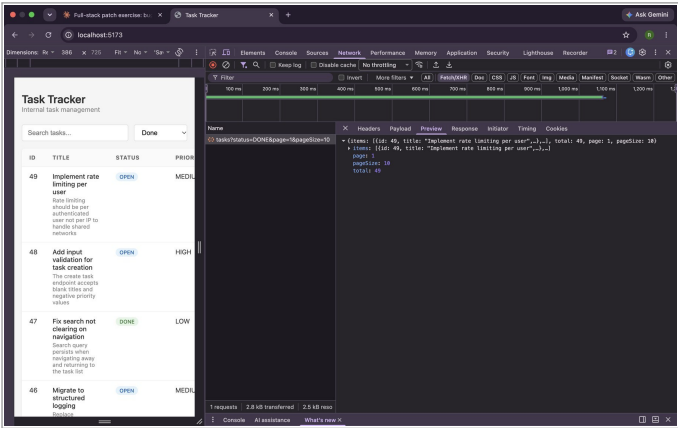
I applied the same fix in the H2 query file and in both Oracle queries. Both Oracle queries matter: one counts rows (the total) and the other returns them, so fixing only one would make the total and the list disagree.

Result

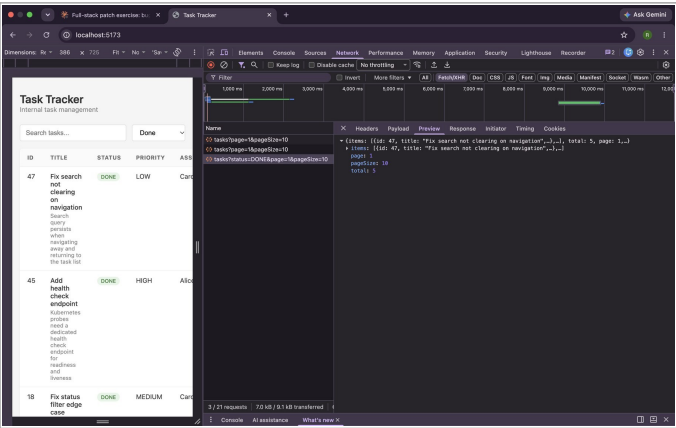
Check (Status = Done)	Before fix	After fix
total in API response	49	5
Rows shown	Mix of OPEN, IN_PROGRESS and DONE	Only DONE (ids 47, 45, 18, ...)

Bug 1: before and after (Status = Done)

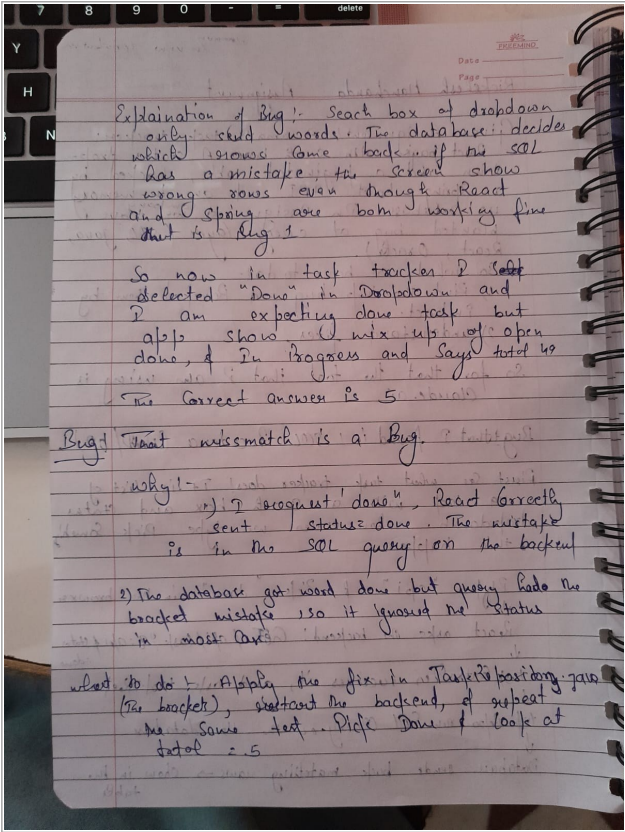
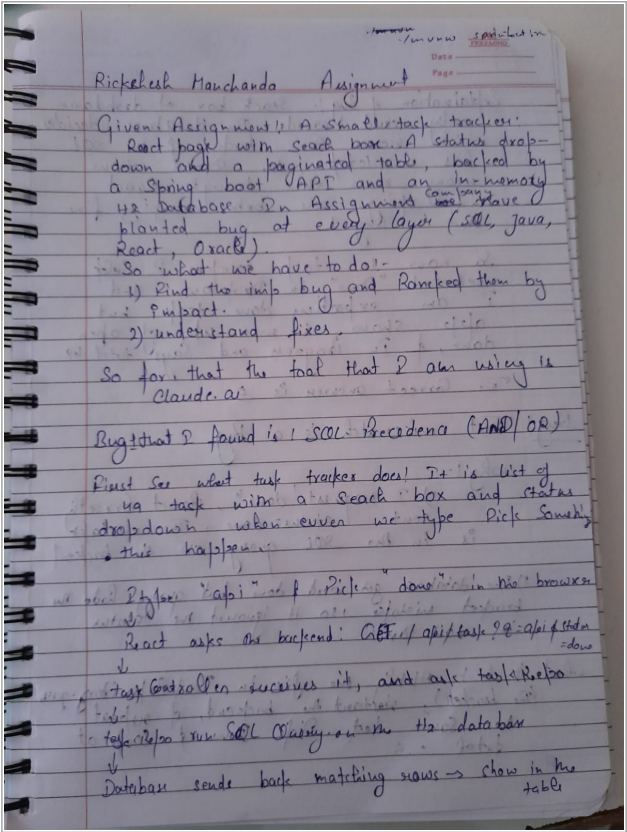
BEFORE the fix: total: 49, and OPEN tasks (49, 48, 46) appear under the Done filter.



AFTER the fix: total: 5, and every row is DONE (47, 45, 18).

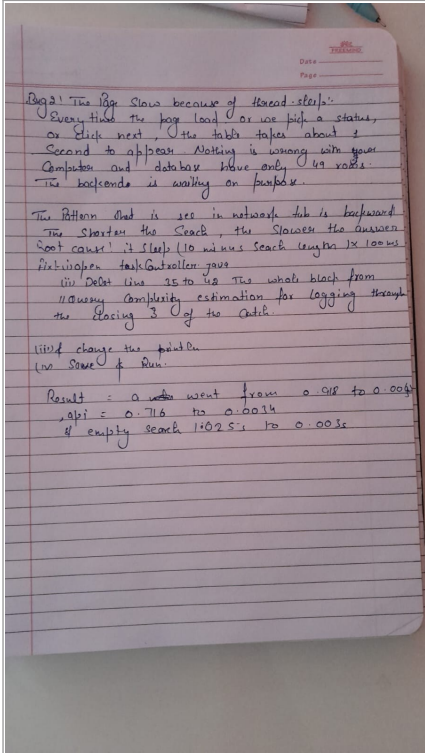


Bug 1: handwritten notes (page 1 left, page 2 right)



Bug 2: Slow Responses Caused by Thread.sleep

Commit message: "Remove artificial Thread.sleep from search endpoint" (f28168e)



Where the bug is

backend/.../TaskController.java, lines 35-42 (backend, Java layer).

How I found it

The page took about a second to show results even though the database has only 49 rows. In the Network tab each request took about 1.0 s (1.02 s, 2.04 s, 1.03 s). I timed the API with curl for four searches and saw a backwards pattern: the shorter the search text, the slower the answer. Reading the controller, I found a Thread.sleep.

Root cause

```
int complexityScore =  
    Math.max(0, 10 - query.length());  
long queryWeight =  
    complexityScore * 100L;  
try { Thread.sleep(queryWeight); }  
catch (InterruptedException e) {...}
```

It sleeps (10 minus search length) x 100 ms, so an empty search waits 1000 ms and 10+ letters wait 0. The comment says it is for logging, but nothing is logged: it only blocks the request thread. The slowest case is the empty search, which is what every user sends when the page first loads.

Fix and why

I deleted the whole sleep block and removed the now-unused complexityScore from the log line. It does no real work, so removing it is the smallest and safest change.

In one line: the backend was waiting on purpose; deleting the sleep removed the delay.

Result (curl, seconds)

Search	Before	After	Faster
(empty)	1.025	0.008	about 130 times
a	0.918	0.0065	about 140 times
api	0.716	0.0034	about 210 times
abcdefghij (10 letters)	0.009	0.0032	no sleep before, so no real change

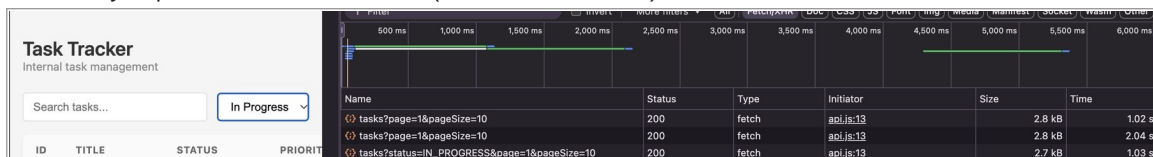
The first request right after a restart took 0.159 s because Spring and Hibernate were warming up. Running the empty search again gave 0.008 s, which is the number I use.

Lesson from debugging

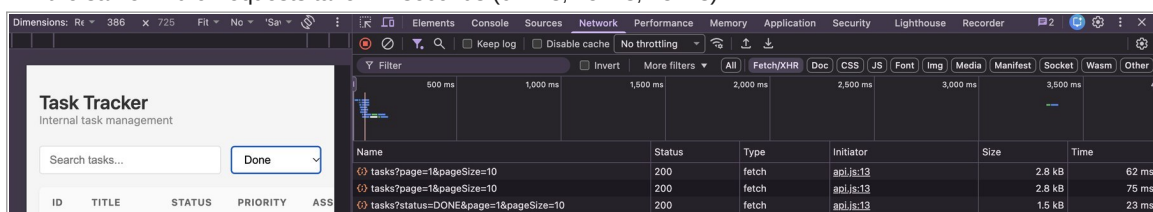
My first timings did not change after the fix because I was still running the old backend from the downloaded zip folder. The startup log showed complexity=10 in every request, which proved the old code was running. After starting the backend from the cloned repo, the times dropped.

Bug 2: before and after (Network tab)

BEFORE the fix: every request takes about 1 second (1.02 s, 2.04 s, 1.03 s).

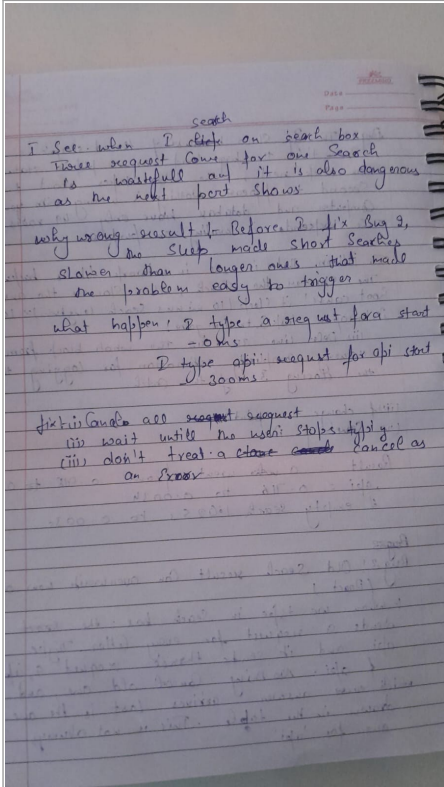


AFTER the fix: the same kind of requests take milliseconds (62 ms, 75 ms, 23 ms).



Bug 3: Search Request on Every Keystroke / Stale Results

Commit message: "Debounce search and cancel stale requests in useTasks"



Where the bug is

frontend/src/hooks/useTasks.js, api.js and App.jsx, plus a new hooks/useDebounce.js (frontend, React layer).

How I found it

I clicked the search box and typed "api" with the Network tab open. One search produced three requests: q=a, q=ap and q=api. That is wasteful, and it is also dangerous, as the next part shows.

Root cause

The search text went straight into the data-loading effect, so every keystroke started a new request, and nothing cancelled the old ones. Responses can come back in a different order than they were sent. Before I fixed Bug 2, the sleep made short searches slower than long ones: the request for "a" (started at 0 ms) took about 0.9 s, while the request for "api" (started at about 300 ms) took about 0.7 s. So "api" could finish first and then the late "a" response would overwrite it, and the table would show results for "a" while the box says "api". After the Bug 2 fix this is hard to trigger by hand, but it is still a real risk on a slow connection.

Fix and why

Three small changes: (i) **debounce** - wait 300 ms after the last keystroke before searching, using a small useDebounce hook; (ii) **cancel** the old request with AbortController when a new one starts, so a stale response can never be applied; (iii) **do not treat a cancel as an error** - an AbortError is ignored, so no false error message is shown.

In one line: wait until the user stops typing, cancel anything still running, and never show a cancel as an error.

Result (typing "api" in the search box)

Check	Before fix	After fix
Search requests for "api"	3 (q=a, q=ap, q=api)	1 (q=api)
Earlier in-flight request	keeps running, can overwrite newer result	cancelled, result ignored

The (canceled) line in the After screenshot is the first page-load request. React StrictMode runs effects twice in development and the cleanup cancels the first one. It is normal, and it proves the cancel code runs.

Bug 3: before and after (Network tab)

BEFORE the fix: typing "api" sends three search requests (q=a, q=ap, q=api).

AFTER the fix: typing "api" sends one search request (q=api); the other lines are page-load requests.

Bug 4: "Loading tasks..." Forever When the Backend Fails

Commit message: "Fix infinite loading state when request fails"

Where the bug is

frontend/src/hooks/useTasks.js, the .catch block (React layer). It works together with the check order in components/TaskTable.jsx.

How I found it

I stopped the backend (Ctrl+C) and reloaded the page. Instead of an error, the page stayed on "Loading tasks..." and never changed.

Root cause

Every request starts with setLoading(true). On success, setLoading(false) runs, but on failure the .catch only did setError(...), so loading stayed true. TaskTable checks loading before error, so it kept showing "Loading tasks..." and the error was never visible. A second problem in the same hook: error was never cleared, so after the backend came back, the old error would show instead of the tasks.

Fix and why

```
setLoading(true);
setError(null); // 1. clear the old error
...
.catch((err) => {
  if (err.name === 'AbortError') return; // newer request running
  setError(err.message);
  setLoading(false); // 2. stop "Loading..." on real failure
});
```

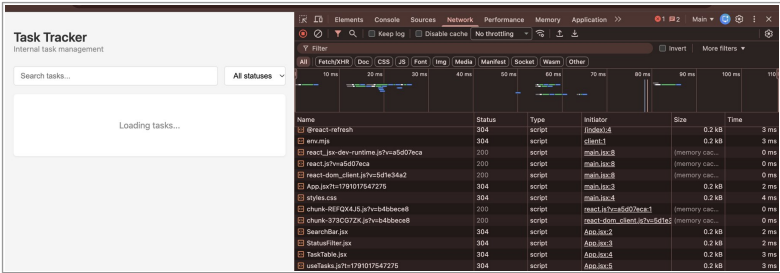
(1) clearing the error at the start of each request fixes the stale error after recovery. (2) setLoading(false) in the catch lets TaskTable reach its error branch. The AbortError case still returns early on purpose: an abort means a newer request is already loading, so loading must stay true (this connects to Bug 3). I fixed the state in the hook instead of reordering TaskTable, because reordering would hide the symptom but leave loading wrong.

Result (backend stopped, page reloaded)

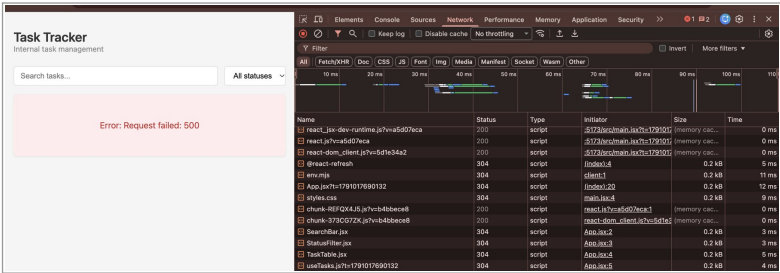
Check	Before fix	After fix
What the page shows	"Loading tasks..." forever	Red "Error: Request failed: 500" message
Request in Network tab	500 (failed), loading never ends	500 (failed), error shown
After the backend is started again	-	Tasks load (200), error gone

Bug 4: before and after (backend stopped)

BEFORE the fix: the request fails with 500 but the page stays on "Loading tasks...".



AFTER the fix: the same 500 now shows "Error: Request failed: 500" in red.



Recovery test: backend started again, then I typed "api". The request returns 200 and the table loads. The (canceled) line is the first page-load request, cancelled by React StrictMode in development.

* ?token=KdBw5CIBRUAZ	101	websocket	client:536	0.0 kB	Pending
api.js?t=1791016237103	304	script	useTasks.js:2	0.2 kB	1 ms
tasks?page=1&pageSize=10	(canceled)	fetch	api.js:13	0.0 kB	1 ms
tasks?page=1&pageSize=10	200	fetch	api.js:13	2.8 kB	189 ms
tasks?q=api&page=1&pageSize=10	200	fetch	api.js:13	2.2 kB	15 ms
22 requests 7.2 kB transferred 1.4 MB resources Finish: 3.11 s DOMContentLoaded: 90 ms Load: 91 ms					

Bug 5: Page Not Reset When Search or Status Changes

Commit message: "Reset to page 1 when search or status changes"

Where the bug is

frontend/src/App.jsx: the page state and the two controls (React layer).

How I found it

I clicked Next twice to reach page 3, then chose Done in the status dropdown. The table said "No tasks found." and the Previous/Next buttons disappeared. In the Network tab the request was tasks?status=DONE&page=3&pageSize=10 and the response was items: [], total: 5, page: 3.

Root cause

page is its own state, and changing the search or the status never set it back to 1. The Done filter only has 5 tasks, which is one page, so page 3 is empty. The pagination buttons are only drawn when totalPages > 1, and here totalPages is 1, so the user had no way to go back.

Fix and why

```
const handleQueryChange = (value) => {
  setQuery(value);
  setPage(1);          // new search -> first page
};
const handleStatusChange = (value) => {
  setStatus(value);
  setPage(1);          // new filter -> first page
};
<SearchBar value={query} onChange={handleQueryChange} />
<StatusFilter value={status} onChange={handleStatusChange} />
```

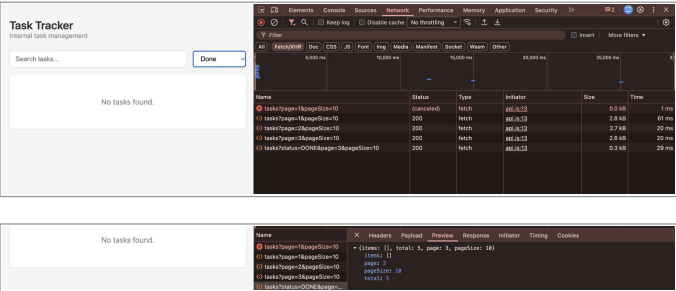
I reset the page at the moment the user changes the search or filter, instead of using an effect. An effect would first send one request with the old page and then another with the new page. This is a small change in one file.

Result (Status = Done, started from page 3)

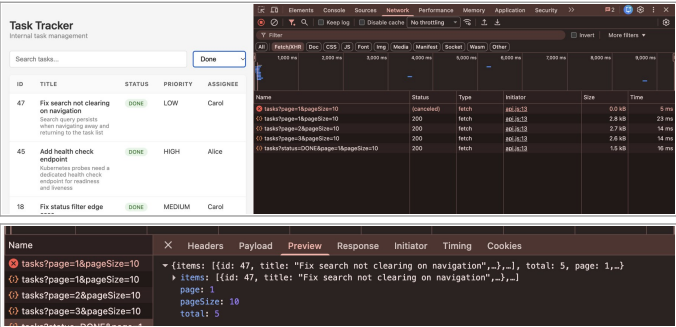
Check	Before fix	After fix
Request sent	status=DONE&page=3	status=DONE&page=1
What the page shows	"No tasks found.", no buttons	5 Done tasks (ids 47, 45, 18, 8, 6)

Bug 5: before and after (Network tab)

BEFORE the fix: Done from page 3 sends page=3 and shows "No tasks found."



AFTER the fix: the same action sends page=1 and shows the five Done tasks.



Summary of fixes (continued)

#	Bug	Where	Result
6	Backend crashed with 500 on invalid status, page or pageSize; pageSize had no upper limit	TaskController.java, searchTasks - Java layer	status=BANANA, page=0, pageSize=-5: 500 changed to 400 with a clear message
7	Paging done in Java memory: every matching row loaded for each page request	TaskRepository.java + TaskController.java - SQL and Java layer	Database now returns only one page (LIMIT/OFFSET) plus a COUNT; responses identical (diff)

Bug 6: Backend Returns 500 on Invalid Input

Commit message: "Validate status, page and pageSize and return 400 instead of 500"

Where the bug is

backend/src/main/java/com/internal/tasktracker/TaskController.java, the searchTasks method (backend, Java layer).

How I found it

I read the controller and saw that status, page and pageSize from the URL are used with no checks. I then called the API with curl using bad values. status=BANANA, page=0 and pageSize=-5 all returned **500** with Spring's default error text, while a valid request (status=done) returned 200.

Root cause

The controller trusts whatever the caller sends, so a mistake by the caller turns into a crash on the server:

- status=BANANA: TaskStatus.valueOf(...) throws an exception for an unknown status.
- page=0: the start index (page - 1) * pageSize becomes negative and subList throws.
- pageSize=-5: the end index lands before the start and subList throws.
- A huge page can overflow the int multiplication and also go negative.
- pageSize has no upper limit, so pageSize=100000 returns every row in one response.

A 500 tells the caller "the server is broken", but here the request was wrong, so the correct answer is 400.

Fix and why

I check the three inputs first and return **400** with a short JSON message when one is wrong: an unknown status (the message lists the allowed values), a page below 1, and a pageSize outside 1 to 100. The status is still read case-insensitively, so status=done keeps working. The start index is calculated with long so a huge page cannot overflow. The React app only sends valid values, so the UI behaves as before.

In one line: check the input first and answer 400 with a clear message, instead of letting bad input crash the server with a 500.

Choice I made: I cap pageSize at 100 and reject bad values instead of silently correcting them. That is a design decision, so I will mention it in NOTES.md.

Result (curl)

Request	Before fix	After fix
status=BANANA	500 Internal Server Error	400 {"error":"Invalid status. Allowed values: [OPEN, IN_PROGRESS, DONE]"} }
page=0	500 Internal Server Error	400 {"error":"page must be 1 or greater"} }
pageSize=-5	500 Internal Server Error	400 {"error":"pageSize must be between 1 and 100"} }
status=done (valid)	200	200 (lowercase still works)

Bug 6: before and after (curl)

BEFORE the fix: the three bad requests return 500 with Spring's default error body; the valid request returns 200.

```
lakshayyadav@lakshays-MacBook-Air superrdev-patch-Assignment % cd backend
lakshayyadav@lakshays-MacBook-Air backend % curl -s -w " " -> %{{http_code}}\n "localhost:8080/api/tasks?status=BANANA"
curl -s -w " " -> %{{http_code}}\n "localhost:8080/api/tasks?page=0"
curl -s -w " " -> %{{http_code}}\n "localhost:8080/api/tasks?pageSize=-5"
curl -s -o /dev/null -w "valid request -> %{{http_code}}\n" "localhost:8080/api/tasks?status=done"
{"timestamp":"2026-10-03T09:09:59.918+00:00","status":500,"error":"Internal Server Error","path":"/api/tasks"} -> 500
{"timestamp":"2026-10-03T09:09:59.935+00:00","status":500,"error":"Internal Server Error","path":"/api/tasks"} -> 500
{"timestamp":"2026-10-03T09:09:59.946+00:00","status":500,"error":"Internal Server Error","path":"/api/tasks"} -> 500
valid request -> 200
lakshayyadav@lakshays-MacBook-Air backend %
```

Anti

AFTER the fix: the same four requests now return 400 with a clear message for each bad input; the valid request still returns 200.

```
lakshayyadav@lakshays-MacBook-Air backend % curl -s -w " " -> %{{http_code}}\n "localhost:8080/api/tasks?status=BANANA"
curl -s -w " " -> %{{http_code}}\n "localhost:8080/api/tasks?page=0"
curl -s -w " " -> %{{http_code}}\n "localhost:8080/api/tasks?pageSize=-5"
curl -s -o /dev/null -w "valid request -> %{{http_code}}\n" "localhost:8080/api/tasks?status=done"
{"error":"Invalid status. Allowed values: [OPEN, IN_PROGRESS, DONE]"} -> 400
{"error":"page must be 1 or greater"} -> 400
{"error":"pageSize must be between 1 and 100"} -> 400
valid request -> 200
lakshayyadav@lakshays-MacBook-Air backend %
```

Bug 7: Paging Done in Memory Instead of the Database

Commit message: "Paginate in the database instead of in memory"

Where the bug is

backend/.../TaskRepository.java (the query) and TaskController.java (backend, SQL and Java layer). This fix builds on Bug 6, which was already applied to the controller.

How I found it

By reading the code, not from a visible symptom. searchTasks asks the database for **all** matching rows and then cuts one page out of the list with subList in Java. With 49 tasks this is invisible, but it does not scale.

Root cause

The paging happens in Java memory, so the cost grows with the size of the table, not with the size of the page. With 100,000 tasks, every click on Next would load all 100,000 rows just to show 10.

Fix and why

The repository now returns a Page with a countQuery, so the database applies LIMIT/OFFSET and returns only the requested page, and a separate COUNT gives the total. I added id DESC as a tie-breaker after created_at DESC, so rows with equal timestamps cannot swap between pages. The JSON the API returns is unchanged.

In one line: let the database return only the page that is needed, instead of loading every row and cutting the page in Java.

Tradeoff: each request now runs two queries (page + count), but both are cheap. The LIKE '%term%' search still scans the table, and an index would not help it. I am leaving that as is.

Bug 7: result

Check	Before fix	After fix
Rows loaded per request	All matching rows, page cut with subList in Java	Only the requested page (LIMIT/OFFSET) plus a separate COUNT for the total
Response for page=2&pageSize=5, status=DONE	Saved to /tmp/before_*.json	Saved to /tmp/after_*.json: identical to the before files
Response for q=api	Saved to /tmp/before_api.json	Saved to /tmp/after_api.json: identical

Bug 7: before and after (proof that the answers did not change)

BEFORE the fix: I saved the API answers for three requests (a page, the Done filter and a search) from the old code.

```
curl -s "localhost:8080/api/tasks?page=2&pageSize=5" > /tmp/before_p2.json
curl -s "localhost:8080/api/tasks?status=DONE" > /tmp/before_done.json
curl -s "localhost:8080/api/tasks?q=api" > /tmp/before_api.json
```

AFTER the fix: I restarted the backend with the new code, saved the same three requests to new files, and compared each pair with diff. Output: **IDENTICAL**, so the fix changed how the data is fetched but not what the API returns.

```
curl -s "localhost:8080/api/tasks?page=2&pageSize=5" > /tmp/after_p2.json
curl -s "localhost:8080/api/tasks?status=DONE" > /tmp/after_done.json
curl -s "localhost:8080/api/tasks?q=api" > /tmp/after_api.json
diff /tmp/before_p2.json /tmp/after_p2.json && \
diff /tmp/before_done.json /tmp/after_done.json && \
diff /tmp/before_api.json /tmp/after_api.json && echo "IDENTICAL"

IDENTICAL
```